

Assignment 2.

1. 1) $eax = 3$, $ebx = 7$ pop
2) $eax = 7$, $ebx = 10$ pop
3) $eax = 3$

Final values: $eax = 3$, $ebx = 10$

2. include Irvine32.inc

• data

num1	SDWORD	1
num2	SDWORD	2
num3	SDWORD	3
num4	SDWORD	4
result	SDWORD	?

• code main proc

```
mov  eax, num1
cmp  eax, num2
jle  second
mov  ebx, num3
cmp  ebx, num4
je   second
add  eax, ebx
mov  result, eax
jmp  end
```

second:

```
mov  eax, num2
cmp  eax, num4
```

```
jne else
mov ebx, num1
add eax, ebx
mov result, eax
jmp end
```

```
else:
mov result, 0
```

```
end:
exit
main endp
end main.
```

3. include Irvine32.inc

.data

```
sum SDWORD 0
i SDWORD 0
count SDWORD 5
array SDWORD 1, 2, 3, 4, 5
```

.code

main proc

L1:

```
mov ecx, i
cmp ecx, count
jge out
```

```

mov esi, i
mov eax, 4
mul esi
mov esi esi, eax
mov eax, array[esi]
cmp eax, 0
jle else
add sum, eax
jmp end

```

```

else:
dec sum

```

```

end:
inc i
jmp L1

```

```

out:
exit

```

```

main endp
end main

```

4. $ax = 0$, $ecx = 10$
- | | | | |
|----|------------|---------------|------------|
| 1) | $ecx = 10$ | $\rightarrow$ | $eax = -1$ |
| 2) | $ecx = 9$ | $\rightarrow$ | $eax = -2$ |
| 3) | $ecx = 8$ | $\rightarrow$ | $eax = -3$ |
| 4) | $ecx = 7$ | $\rightarrow$ | $eax = -4$ |
| 5) | $ecx = 6$ | $\rightarrow$ | $eax = -5$ |

$$6) \quad ecx = 5 \longrightarrow ax = -6$$

$$7) \quad ecx = 4 \longrightarrow ax = -7$$

$$8) \quad ecx = 3 \longrightarrow ax = -8$$

$$9) \quad ecx = 2 \longrightarrow ax = -9$$

$$10) \quad ecx = 1 \longrightarrow ax = -10$$

$$11) \quad ecx = 0 \longrightarrow ax = -10$$

Final value of $ax = -10$.

5. 1) MOV AL, 0D4h

$$AL = 0D4h = 1101\ 0100b$$

2) SHL AL, 3

$$AL = 0A0h = 1010\ 0000b$$

3) MOV AL, 0D4h

$$AL = 0D4h = 1101\ 0100b$$

4) SAL AL, 3

$$AL = 0A0h = 1010\ 0000b$$

5) MOV AL, 0D4h

$$AL = 0D4h = ~~1010~~ 1101\ 0100b$$

6) ROL AL, 1

$$AL = 0A9h = 1010\ 1001b$$

7) MOV AL, 0D4h

$$AL = 0D4h = 1101\ 0100b$$

8) RCR AL, 3

$$AL = 3Ah = 0011\ 1010b$$

6. include Irvine32.inc

• data

op1	SDWORD	10
op2	SDWORD	20
x	SDWORD	30
y	SDWORD	40
z	SDWORD	0

• code

main proc

L1:

```
mov     eax, op1
cmp     eax, op2
jg      out out
cmp     eax, x
jl      else
cmp     eax, y
jg      else
mov     ebx, z
add     ebx, 10
mov     z, ebx
jmp     end
```

else:

```
mov     ebx, z
sub     ebx, 10
mov     z, ebx
```

```
end:  
dec op1  
jmp L1
```

```
out:  
exit  
main endp  
end main.
```

7. include Irvine32.inc

• data

```
msg1 byte "The value is positive.", 0  
msg2 byte "The value is negative.", 0  
msg3 byte "The value is zero.", 0
```

• code

```
; assuming AL contains signed byte value  
cmp al, 0  
jg positive  
jl negative  
mov edx, offset msg3  
jmp display
```

positive:

```
mov edx, offset msg1  
jmp display
```

negative:

mov ecx, offset msg2

display:

call writeString

call crlf

exit

main endp

end main

8. include Irvine32.inc

• data

num1 SDWORD 10

num2 SDWORD 20

result ~~SDWORD~~ SDWORD ?

• code

main proc

movsx ax, num1

imul num2

push edx

push eax

pop eax

mov result, eax

call writeInt

exit

main endp

end main

9. include Irvine32.inc

• data

array WORD 10, 60, 20, 33, 72, 89, 45, 65, 72, 18
sample WORD 50
ArraySize WORD ?
index WORD 0
sum WORD 0

• code

main proc

mov eax, sizeof array

mov ebx, sizeof sample

div ebx

mov ArraySize, ax

L1:

cmp ax, index

jge out

mov bx, array[index]

cmp bx, sample

jg end

add sum, bx

end:

~~add~~ ~~index~~ ~~add~~ mov cx, index

add cx, 2

mov index, cx

jmp L1

out:

exit

main endp

end main

10. a) include irvine32.inc

• data

• code

main proc

; assuming eax contains the value to be multiplied

mov ebx, eax

shl eax, 5

shl ebx, 2

add eax, ebx

exit

main endp

end main

b) 1) $eax = 2$, $ebx = 2$

2) Shifting: 2×2^5 , $2 \times 2^2 = 64$, 8

3) Addition: $64 + 8 = 72$

Final value in $eax = 72$